



CONNECT FOUR

A browser-based game built with vanilla HTML, CSS & JavaScript



Project Overview

● Why Connect Four?

One of my favorite games -- and a clean, self-contained scope to practice DOM manipulation and game-state logic from scratch.

● What It Does

Classic 2-player Connect Four in the browser: drop discs, connect 4 to win. Play a friend, or a simple computer opponent.

● How It Works

The board is a 6×7 array. Every click updates that array, re-renders the DOM from it, then checks 4 directions for a win.

● What's Next

AI difficulty levels, sound & animation, online multiplayer, undo, match history, and a best-of-three mode.



Built With

No frameworks, no build tools -- just the fundamentals covered in lecture

HTML

Semantic structure, accessible
instructions & controls

CSS

Grid for the board, Flexbox for
layout, custom properties for
theming

JS

ES6 DOM manipulation, game
logic, computer opponent

LS

Web Storage API -- scores,
names, and leaderboard persist
on reload



Features



PvP & PvC Modes

Two players, or challenge a computer opponent



Timer & Pause

Live round timer that pauses with the game



Scoreboard

Wins, ties, and a persistent leaderboard by name



Win Detection

Checks all 4 directions, highlights the winning line



Dark / Light Theme

One-click toggle, remembered between visits



Responsive Design

Full layout adapts for desktop, tablet, and mobile

The Code: Rendering the Board

From js/script.js -- the board is redrawn from state, never edited by hand



```
const createBoard = () => Array.from({ length: ROWS }, () => Array(COLUMNS).fill(''));  
// a fresh 6x7 grid -- every cell starts as an empty string  
  
function renderBoard() {  
  const fragment = document.createDocumentFragment(); // build off-screen, insert once  
  cells.forEach((row, rowIndex) => row.forEach((player, columnIndex) => {  
    const cell = document.createElement('div'); // one <div> per board cell  
    cell.className = 'cell';  
    cell.dataset.row = rowIndex; // store position for click handling  
    cell.dataset.col = columnIndex;  
    if (player) cell.classList.add(player); // color it if a disc is there  
    if (winningCells.some(([r, c]) => r === rowIndex && c === columnIndex))  
      cell.classList.add('win'); // highlight if part of the win  
    fragment.appendChild(cell);  
  }));  
  ui.board.replaceChildren(fragment); // one DOM update instead of 42 separate ones  
}
```

the board is never edited by hand -- every render rebuilds all 42 cells from the cells array, so the DOM can't drift out of sync.

The Code: Event Listeners

From js/script.js -- every control on the page, wired up in one place



```
ui.board.addEventListener('click', (event) => {
  const cell = event.target.closest('.cell'); // find which cell was clicked
  if (!cell || gameOver || isPaused || isComputerThinking) return; // ignore bad clicks
  if (playMove(Number(cell.dataset.col))) scheduleComputerMove(); // drop in that column
});

ui.pause.addEventListener('click', togglePause);
ui.restart.addEventListener('click', restartGame);
ui.reset.addEventListener('click', resetEverything);

ui.theme.addEventListener('click', () => {
  const isLight = !document.body.classList.contains('light');
  applyTheme(isLight); // flip the theme
  localStorage.setItem(THEME_KEY, isLight ? 'light' : 'dark'); // remember it
});

ui.modes.forEach((button) =>
  button.addEventListener('click', () => setMode(button.dataset.mode))); // PvP / PvC
ui.inputs.Player1.addEventListener('input', () => updatePlayerName('Player1'));
ui.inputs.Player2.addEventListener('input', () => updatePlayerName('Player2'));
```

one listener per control, all wired once at the bottom of the file -- the board's click listener is the only one that has to work out which cell was clicked.

The Code: Win Detection

★ Hardest Part

From `js/script.js` -- scans outward in 4 directions from the disc that was just placed

```
const DIRECTIONS = [[0, 1], [1, 0], [1, 1], [1, -1]];
// each pair is [rowStep, colStep] -> right, down, diagonal \, diagonal /

function getLine(row, column, rowStep, columnStep, player) {
  const line = [];
  for (
    let r = row + rowStep, c = column + columnStep; // start 1 step from the disc
    r >= 0 && r < ROWS && c >= 0 && c < COLUMNS // stay inside the board...
    && cells[r][c] === player; // ...and keep matching this player
    r += rowStep, c += columnStep // step further in this direction
  ) line.push([r, c]); // record every matching cell
  return line; // matches in ONE direction only
}

function findWinningCells(row, column, player) {
  for (const [rowStep, columnStep] of DIRECTIONS) {
    const line = [[row, column], // the disc just placed
      ...getLine(row, column, rowStep, columnStep, player), // + matches going forward
      ...getLine(row, column, -rowStep, -columnStep, player)]; // + matches going backward
    if (line.length >= 4) return line; // 4+ in one full line -> win, return those cells
  }
  return []; // no direction reached 4 discs -> not a win yet
}
```

check the 4 lines running through the last disc placed -- if any one hits 4+ in a row, that's a win.

The Code: Computer Opponent

From js/script.js -- reuses findWinningCells to test moves before committing to one



```
function getComputerColumn() {
  const valid = Array.from({ length: COLUMNS }, (_, column) => column)
    .filter((column) => getAvailableRow(column) !== -1);    // only columns with room left

  const findTacticalMove = (player) => valid.find((column) => {
    const row = getAvailableRow(column);
    cells[row][column] = player;                          // temporarily place the disc
    const wins = findWinningCells(row, column, player).length > 0; // would this move win?
    cells[row][column] = '';                               // undo it -- this was just a test
    return wins;                                           // true = this column works
  });

  return findTacticalMove('Player2')                      // 1. take my own winning move
    ?? findTacticalMove('Player1')                        // 2. else block the human's win
    ?? valid[Math.floor(Math.random() * valid.length)];    // 3. else play any legal column
}
```

Priority order: take a winning move → block the human's winning move → play a random valid column.

Anticipated question -- “Does it ever lose on purpose?” No -- it always takes a win or a block first; random is only the last resort.

The Code: Saving Game State

From js/script.js -- the Web Storage API keeps progress across a refresh



```
function saveState() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify({
    cells, currentPlayer, gameOver, scores, seconds, gameMode, playerNames,
    statusText: ui.statusText.textContent, statusTone: ui.status.className
  })); // one snapshot of everything needed to resume exactly where you left off
}

function loadState() {
  const saved = getSavedData(STORAGE_KEY, null);
  if (!saved) return false; // nothing saved yet -- start fresh
  cells = saved.cells || createBoard();
  currentPlayer = saved.currentPlayer || 'Player1';
  gameOver = Boolean(saved.gameOver);
  scores = { ...DEFAULT_SCORES, ...saved.scores }; // fill in any missing keys safely
  seconds = saved.seconds || 0;
  gameMode = saved.gameMode || 'pvp';
  setStatus(saved.statusText || '', saved.statusTone || '');
  return true; // tells the caller a saved game WAS restored
}
```

saveState runs after every move; loadState runs once on page load -- if nothing's saved yet, the game just starts clean instead of erroring.



The Code: Dark / Light Theming

JS toggles one class; CSS custom properties do the rest of the work



```
function applyTheme(isLight) {
  document.body.classList
    .toggle('light', isLight);    // add/remove
    .light on <body>
  if (ui.moon)
    ui.moon.style.display =
      isLight ? 'none' : 'block';
  if (ui.sun)
    ui.sun.style.display =
      isLight ? 'block' : 'none';  // swap the
    icon
}
```

js/script.js



```
:root{
  --text:#F8FAFF;      /* dark theme */
  --muted:rgba(248,250,255,.78);
}

body.light{
  --text:#181c2b;      /* light theme */
  --muted:rgba(24,28,43,.8);
}

/* every rule below just reads
   var(--text) / var(--muted) */
```

css/style.css

toggling one class swaps every CSS variable at once -- no JavaScript ever touches an individual color.



The Code: Responsive Design

From css/style.css -- 3 breakpoints reflow the layout for tablet and phone



```
@media (max-width:1024px){  /*  
  tablet/laptop */  
  #gameArea{  
    flex-direction:column;  /*  
  stack panels */  
  }  
  #leaderboardPanel{  
    width:100%;  /* fill row width  
  */  
  }  
}
```

Tablet / Laptop

≤ 1024px



```
@media (max-width:768px){  /*  
  portrait phones */  
  #header{  
    flex-direction:column;  /*  
  stack header */  
    text-align:center;  
  }  
  #setupPanel{  
    flex-direction:column;  /*  
  stack setup */  
  }  
}
```

Large Phone / Tablet

≤ 768px



```
@media (max-width:480px){  /*  
  phones */  
  #gameBoard{  
    grid-template-columns:  
      repeat(7,32px);  /* was 56px  
  */  
  }  
  .cell{  
    width:32px;  
    height:32px;  /* discs shrink  
  */  
  }  
}
```

Phone

≤ 480px



What Was Hardest -- and Why

● Win detection

Checking exactly 4 lines through the last move, instead of rescanning all 42 cells every turn -- the logic took a few tries to get right.

● A believable opponent in ~15 lines

Reusing the same win-check function on hypothetical moves meant no second, duplicate algorithm to maintain.

● State that never drifts

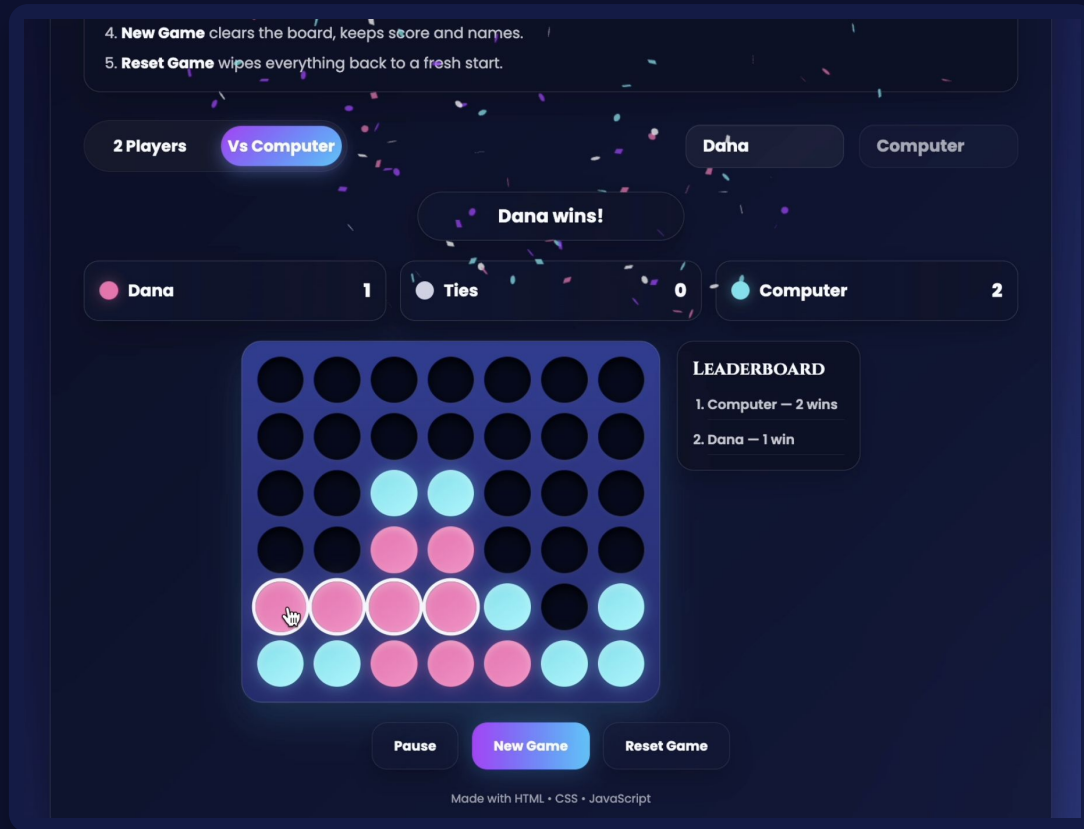
The DOM is rebuilt from the cells array on every render, so game state and what's on screen can never disagree.

● Accessible by design

Went back through the palette to check WCAG AA contrast on a custom dark/light glassmorphism theme.



Live Demo & What's Next



A win, celebrated

dana12812.github.io/connect4-game

Stretch Goals

- AI difficulty levels
- Sound effects
- Disc-drop animation
- Online multiplayer
- Undo move
- Match history
- Player statistics
- Best-of-three mode



Thank You

Questions?

github.com/dana12812/connect4-game • dana12812.github.io/connect4-game